

FreeLedger

Test Case Suite

Backend, Database, Storage, and Security

Author: Pawan Poudel

Date: 03 July 2026

Document Version: 1.0

Document Information

Project Scope	FreeLedger — Decentralized Freelance Protocol 35 test cases across Smart Contract Engine (M2), Workflow API (M5), Wallet Auth (M1), Database and Security (M4)
Derived From	Individual Capstone Report (Chapter 3 & 4)
Mapped To Framework	22 Functional & Non-Functional Requirements pytest-asyncio, httpx.AsyncClient, unittest.mock

through dispute resolution, mapped to the system's functional and non-functional requirements.

Contents

1	Requirements Traceability Matrix	2
1.1	Functional Requirements (from Chapter 3.3.1)	2
1.2	Non-Functional Requirements (from Chapter 3.3.2)	2
2	Module M1 — Wallet Authentication	3
2.1	TC ID: TC-M1-01 — MetaMask Auth: Nonce Challenge and Signature Verification	3
2.2	TC ID: TC-M1-02 — MetaMask Auth: Nonce Uniqueness Per Request	4
2.3	TC ID: TC-M1-03 — Replay Attack: Nonce Invalidation After Use	5
2.4	TC ID: TC-M1-04 — Signature Mismatch Rejection	6
3	Module M2 — Smart Contract Engine	7
3.1	TC ID: TC-M2-01 — Contract Creation: IPFS Upload + On-Chain Deployment	7
3.2	TC ID: TC-M2-02 — Contract Creation: IPFS Failure Falls Back to Contract CID	8
3.3	TC ID: TC-M2-03 — Contract Creation: Milestone Sum Mismatch Rejected	9
3.4	TC ID: TC-M2-04 — Contract Creation: Non-Client Blocked	9
3.5	TC ID: TC-M2-05 — Contract Creation: IPFS Timeout Graceful Fallback	10
3.6	TC ID: TC-M2-06 — Contract Creation: Full Failure Still Persists Contract	11
3.7	TC ID: TC-M2-07 — Milestone Submission: Freelancer Submits Deliverable	12
3.8	TC ID: TC-M2-08 — Milestone Approval: Client Approves, ETH Released	13
3.9	TC ID: TC-M2-09 — Milestone Approval: Double Approve Rejected	14
3.10	TC ID: TC-M2-10 — Milestone Approval: All Milestones Complete Contract	15
4	Module M3 — Database & State Management	16
4.1	TC ID: TC-M3-01 — Contract State Transition: Active → Completed	16
4.2	TC ID: TC-M3-02 — Milestone State Machine: Pending → Submitted → Approved	17
4.3	TC ID: TC-M3-03 — Cannot Submit Approved Milestone	17
4.4	TC ID: TC-M3-04 — Cannot Submit Already Submitted Milestone	18
4.5	TC ID: TC-M3-05 — Approve Pending Milestone Rejected	18
4.6	TC ID: TC-M3-06 — Dispute State: Active → Disputed	19
4.7	TC ID: TC-M3-07 — Cannot Dispute Completed Contract	19
4.8	TC ID: TC-M3-08 — Duplicate Dispute Blocked	20
5	Module M4 — Security & RBAC	21
5.1	TC ID: TC-M4-01 — Unauthenticated Request Blocked	21
5.2	TC ID: TC-M4-02 — Freelancer Cannot Approve Milestones	21
5.3	TC ID: TC-M4-03 — Client Cannot Submit Milestones	22
5.4	TC ID: TC-M4-04 — Non-Freelancer Cannot Submit Milestones	22
5.5	TC ID: TC-M4-05 — Freelancer Cannot Resolve Disputes	23
5.6	TC ID: TC-M4-06 — Client Cannot Resolve Disputes	23
5.7	TC ID: TC-M4-07 — Freelancer Raises Dispute	24
5.8	TC ID: TC-M4-08 — Password Hashing Verification	24

6	Module M5 — Workflow API (FastAPI)	25
6.1	TC ID: TC-M5-01 — Health Check Endpoint	25
6.2	TC ID: TC-M5-02 — Create Contract API: Full Integration	25
6.3	TC ID: TC-M5-03 — Submit Milestone API: Full Integration	26
6.4	TC ID: TC-M5-04 — Approve Milestone API: Full Integration	26
6.5	TC ID: TC-M5-05 — Raise Dispute API: Full Integration	27
6.6	TC ID: TC-M5-06 — Admin Resolve Dispute: Release to Freelancer . . .	28
6.7	TC ID: TC-M5-07 — Admin Resolve Dispute: Refund to Client	29
6.8	TC ID: TC-M5-08 — Concurrent Approvals: Race Condition Prevention	30
6.9	TC ID: TC-M5-09 — Double Approve: Sequential Rejection	30
6.10	TC ID: TC-M5-10 — Contract Terms IPFS Upload with Fallback	31
7	Test Execution Summary	32
7.1	Test Case Distribution	32
7.2	Priority Distribution	32
7.3	Technique Distribution	32
7.4	Requirements Coverage	32

1 Requirements Traceability Matrix

1.1 Functional Requirements (from Chapter 3.3.1)

Req ID	Module	Description
FR-DS-01	M3	Project metadata, milestones, and user data stored in PostgreSQL
FR-DS-02	M3	Consistency between stored data and system states maintained
FR-IPFS-01	M5	Users upload project files to IPFS
FR-IPFS-02	M5	CID generated for each uploaded file
FR-IPFS-03	M5	Uploaded files linked to corresponding milestones
FR-SYNC-01	M3	Data synchronized across PostgreSQL, IPFS, and blockchain
FR-SYNC-02	M3	Updates propagated correctly across all storage layers
FR-BE-01	M5	Backend handles user requests and coordinates DB, IPFS, blockchain
FR-BE-02	M5	Async processing to handle blockchain/IPFS delays
FR-SEC-01	M4	Role-based access control (RBAC) enforced
FR-SEC-02	M4	All incoming requests validated before processing
FR-AUTH-01	M1	MetaMask wallet authentication with nonce-based challenge
FR-AUTH-02	M1	Email/password authentication as fallback
FR-CON-01	M2	Smart contract deployed on-chain with milestone structure
FR-CON-02	M2	Contract terms uploaded to IPFS with CID reference
FR-MILE-01	M2	Milestone submission records deliverable CID on-chain
FR-MILE-02	M2	Milestone approval triggers ETH payout with platform fee
FR-DISP-01	M2	Dispute can be raised by contract party, locks escrow
FR-DISP-02	M2	Admin-only dispute resolution with refund/release

1.2 Non-Functional Requirements (from Chapter 3.3.2)

Req ID	Module	Description
NFR-PERF-01	M5	Database queries executed with low latency
NFR-PERF-02	M5	Concurrent requests handled without blocking
NFR-SEC-01	M4	Sensitive data protected using salting and hashing
NFR-SEC-02	M4	Middleware enforced for secure API access
NFR-SCAL-01	M3	Blockchain storage reduced by storing only CID on IPFS
NFR-SCAL-02	M5	Backend scales with user and data loads
NFR-AVAIL-01	M3	IPFS files pinned for availability
NFR-AVAIL-02	M5	System remains operational if individual nodes fail
NFR-REL-01	M3	Data consistent between off-chain and on-chain components
NFR-REL-02	M3	System failures do not result in data loss or inconsistency

2 Module M1 — Wallet Authentication

2.1 TC ID: TC-M1-01 — MetaMask Auth: Nonce Challenge and Signature Verification

Field	Value
Req Mapping	FR-AUTH-01, FR-SEC-02
Priority	High
Technique	Functional — Valid partition
Precondition	MetaMask installed and unlocked; backend running on port 8000; Redis available for nonce storage; client wallet seeded in PostgreSQL (wallet_address = 0x70997970c51812dc3a010c7d01b50e0d17dc79c8)
Test Steps	1. Frontend calls GET /api/auth/wallet/challenge?address=0x7099...c8 2. Backend generates 64-character hex nonce 3. Nonce stored in Redis with key nonce:0x7099...c8 4. Frontend calls POST /api/auth/wallet/login with address, signature, and role 5. Backend recovers public key from signature using <code>eth_account.Account.recover_message</code> 6. Backend compares recovered address to claimed address
Test Data	• Valid Ethereum address (checksummed): 0x70997970c51812dc3a010c7d01b50e0d17dc79c8 • Valid EIP-191 signature (65 bytes, 130 hex chars) • Role: "client"
Expected Result	• HTTP 200 with JSON body • Response contains <code>access_token</code> and <code>token_type: "bearer"</code> • Response contains <code>user.id</code> matching seeded client • Nonce deleted from Redis after successful verification
Pass/Fail Criteria	• PASS if HTTP 200 + token returned + nonce consumed from Redis • FAIL if HTTP 401/403 or nonce not deleted

2.2 TC ID: TC-M1-02 — MetaMask Auth: Nonce Uniqueness Per Request

Field	Value
Req Mapping	FR-AUTH-01, FR-SEC-02
Priority	High
Technique	Functional — Deterministic uniqueness
Precondition	Backend running; Redis available; client wallet address known
Test Steps	1. Call GET <code>/api/auth/wallet/challenge?address=0x7099...c8</code> — store nonce as <code>nonce_1</code> 2. Call GET <code>/api/auth/wallet/challenge?address=0x7099...c8</code> — store nonce as <code>nonce_2</code> 3. Assert <code>nonce_1 != nonce_2</code> 4. Assert both nonces are 64 characters long 5. Assert both nonces are valid hex strings
Test Data	• Same wallet address used for both requests • No authentication header required for challenge endpoint
Expected Result	• Each challenge returns a different 64-character hex nonce • Second nonce overwrites first in Redis (key is same)
Pass/Fail Criteria	• PASS if two consecutive challenges produce distinct nonces of length 64 • FAIL if nonces are identical or malformed

2.3 TC ID: TC-M1-03 — Replay Attack: Nonce Invalidation After Use

Field	Value
Req Mapping	FR-AUTH-01, NFR-SEC-02
Priority	High
Technique	Security — Replay attack prevention
Precondition	Backend running; Redis available; client wallet address seeded
Test Steps	1. Call <code>GET /api/auth/wallet/challenge?address=...</code> to generate nonce 2. Call <code>POST /api/auth/wallet/login</code> with valid signature — expect HTTP 200 3. Nonce deleted from Redis after first successful login 4. Call <code>POST /api/auth/wallet/login</code> again with same signature — expect HTTP 401 5. Assert error code is "CHALLENGE_EXPIRED"
Test Data	• Same address, signature, and role for both login attempts • Second attempt reuses already-consumed nonce
Expected Result	• First login succeeds (HTTP 200) • Second login fails (HTTP 401) with CHALLENGE_EXPIRED
Pass/Fail Criteria	• PASS if replay attempt returns 401 and nonce is absent from Redis • FAIL if second login succeeds (replay vulnerability)

2.4 TC ID: TC-M1-04 — Signature Mismatch Rejection

Field	Value
Req Mapping	FR-AUTH-01, FR-SEC-01
Priority	High
Technique	Security — Invalid signature partition
Precondition	Backend running; Redis available; client wallet seeded
Test Steps	1. Call GET /api/auth/wallet/challenge?address=0x7099...c8 2. Mock <code>eth_account.Account.recover_message</code> to return wrong address (0x000...001) 3. Call POST /api/auth/wallet/login with mismatched signature 4. Assert HTTP 401 5. Assert error code is "SIGNATURE_MISMATCH"
Test Data	• Claimed address: 0x70997970c51812dc3a010c7d01b50e0d17dc79c8 • Recovered address (mocked): 0x00000000000000000000000000000000 • Signature: 0xabab...ab (65 bytes, value irrelevant due to mock)
Expected Result	• HTTP 401 with SIGNATURE_MISMATCH error • No session created; nonce not consumed
Pass/Fail Criteria	• PASS if mismatched signature returns 401 • FAIL if request succeeds or returns non-401 status

3 Module M2 — Smart Contract Engine

3.1 TC ID: TC-M2-01 — Contract Creation: IPFS Upload + On-Chain Deployment

Field	Value
Req Mapping	FR-CON-01, FR-CON-02, FR-IPFS-02, FR-BE-01
Priority	High
Technique	Functional — Full integration path
Precondition	Client authenticated; job <code>job_001</code> seeded; freelancer <code>usr_freel1</code> seeded; IPFS mock returning <code>{cid, size}</code> ; blockchain mock returning <code>{on_chain_id, tx_hash}</code>
Test Steps	1. Client calls <code>POST /api/contracts</code> with <code>job_id</code>, <code>freelancer_id</code>, <code>title</code>, <code>total_amount</code>, <code>milestones</code> array 2. Backend uploads contract terms to IPFS via <code>upload_contract_terms()</code> — returns CID 3. Backend calls <code>create_contract_on_chain()</code> via <code>asyncio.to_thread</code> — returns <code>on_chain_id</code> 4. Contract saved to PostgreSQL with <code>terms_cid</code>, <code>on_chain_id</code>, <code>status = pending_signatures</code> 5. Milestones saved to PostgreSQL with <code>status pending</code>
Test Data	• <code>job_id</code>: "job_001" • <code>freelancer_id</code>: "usr_freel1" • <code>title</code>: "Web3 DApp" • <code>total_amount</code>: 1000.0 • <code>milestones</code>: [{"Frontend", 500.0}, {"Backend", 500.0}]
Expected Result	• HTTP 201 with <code>ContractResponse</code> • <code>terms_cid == "QmTestHash123"</code> (mocked IPFS CID) • <code>on_chain_id == 42</code> (mocked blockchain ID) • <code>status == "pending_signatures"</code> • Two milestones created in PostgreSQL
Pass/Fail Criteria	• PASS if contract persisted with IPFS CID + on-chain ID + correct milestone count • FAIL if any field missing or status incorrect

3.2 TC ID: TC-M2-02 — Contract Creation: IPFS Failure Falls Back to Contract CID

Field	Value
Req Mapping	FR-IPFS-02, NFR-REL-02, NFR-AVAIL-01
Priority	High
Technique	Edge case — Degraded mode
Precondition	Client authenticated; IPFS mock raises <code>Exception("IPFS down")</code> ; blockchain mock succeeds
Test Steps	1. Client calls <code>POST /api/contracts</code> with valid payload 2. <code>upload_contract_terms()</code> raises <code>Exception</code> 3. Backend catches exception, falls back to <code>terms_cid = f"contract_{contract_id}"</code> 4. Contract still created on-chain with fallback CID
Test Data	• IPFS: mocked to raise <code>Exception</code> • Blockchain: mocked to return <code>on_chain_id = 99</code> • Milestones: <code>[{"Work", 500.0}]</code>
Expected Result	• HTTP 201 (contract still created) • <code>terms_cid</code> starts with <code>"contract_"</code> • Contract persisted in PostgreSQL despite IPFS failure
Pass/Fail Criteria	• PASS if contract created with fallback CID when IPFS is unavailable • FAIL if HTTP 500 or contract not persisted

3.3 TC ID: TC-M2-03 — Contract Creation: Milestone Sum Mismatch Rejected

Field	Value
Req Mapping	FR-DS-02, FR-SEC-02
Priority	High
Technique	Negative testing — Data integrity
Precondition	Client authenticated
Test Steps	1. Client calls <code>POST /api/contracts</code> with <code>total_amount = 1000</code> but milestones summing to 600 2. Backend validates milestone amounts sum to <code>total_amount</code> 3. Assert HTTP 400
Test Data	• <code>total_amount</code>: 1000.0 • <code>milestones</code>: [{“A”, 300.0}, {“B”, 300.0}] — <code>sum = 600 ≠ 1000</code>
Expected Result	• HTTP 400 with "MILESTONE_SUM_MISMATCH" error code • No contract created in PostgreSQL or on-chain
Pass/Fail Criteria	• PASS if mismatched amounts return 400 • FAIL if contract created despite mismatch

3.4 TC ID: TC-M2-04 — Contract Creation: Non-Client Blocked

Field	Value
Req Mapping	FR-SEC-01, FR-SEC-02
Priority	High
Technique	Security — RBAC restriction
Precondition	Freelancer authenticated (<code>role = freelancer</code>)
Test Steps	1. Freelancer calls <code>POST /api/contracts</code> with valid payload 2. Backend checks <code>current_user.role != client</code> 3. Assert HTTP 403
Test Data	• Auth token for <code>usr_freel1</code> (freelancer role) • Payload: {title: “Nope”, <code>total_amount</code>: 100.0}
Expected Result	• HTTP 403 with "CLIENT_ONLY" error code • No contract created
Pass/Fail Criteria	• PASS if freelancer receives 403 when attempting contract creation • FAIL if freelancer can create contracts

3.5 TC ID: TC-M2-05 — Contract Creation: IPFS Timeout Graceful Fallback

Field	Value
Req Mapping	FR-IPFS-02, NFR-REL-02, NFR-AVAIL-01
Priority	Medium
Technique	Edge case — Network timeout
Precondition	Client authenticated; IPFS mock raises <code>httpx.TimeoutException</code>
Test Steps	1. Client calls <code>POST /api/contracts</code> 2. <code>upload_contract_terms()</code> raises <code>TimeoutException</code> 3. Backend catches timeout, uses fallback CID 4. Contract persisted on-chain with fallback <code>terms_cid</code>
Test Data	• IPFS: mocked to raise <code>httpx.TimeoutException("timeout")</code> • Blockchain: mocked to return <code>on_chain_id = 1</code>
Expected Result	• HTTP 201 • <code>terms_cid</code> starts with <code>"contract_"</code>
Pass/Fail Criteria	• PASS if timeout results in fallback CID, not HTTP 500 • FAIL if timeout propagates as unhandled exception

3.6 TC ID: TC-M2-06 — Contract Creation: Full Failure Still Persists Contract

Field	Value
Req Mapping	FR-BE-01, NFR-REL-02
Priority	Medium
Technique	Edge case — Partial failure recovery
Precondition	Client authenticated; both IPFS and blockchain mocks raise exceptions
Test Steps	1. Client calls <code>POST /api/contracts</code> 2. <code>upload_contract_terms()</code> raises Exception 3. <code>asyncio.to_thread(create_contract_on_chain)</code> raises Exception 4. Backend catches both, persists contract with <code>on_chain_id = None</code> and fallback CID
Test Data	• IPFS: <code>Exception("IPFS")</code> • Blockchain: <code>Exception("Chain")</code>
Expected Result	• HTTP 201 • <code>on_chain_id</code> is <code>None</code> • <code>terms_cid</code> starts with <code>"contract_"</code>
Pass/Fail Criteria	• PASS if contract persisted in PostgreSQL even when both IPFS and chain fail • FAIL if HTTP 500 or contract not saved

3.7 TC ID: TC-M2-07 — Milestone Submission: Freelancer Submits Deliverable

Field	Value
Req Mapping	FR-MILE-01, FR-IPFS-03
Priority	High
Technique	Functional — State transition
Precondition	Contract <code>ct_001</code> active; milestone <code>ms_000</code> in <code>pending</code> state; freelancer authenticated; blockchain mock returns tx hash
Test Steps	1. Freelancer calls <code>POST /api/contracts/ct_001/milestones/0/submit</code> 2. Backend verifies milestone is <code>pending</code> 3. Backend calls <code>submit_milestone_on_chain()</code> (mocked) 4. Milestone status updated to <code>submitted</code> 5. <code>deliverable_cid</code> and <code>submission_notes</code> saved 6. Notification sent to client
Test Data	• <code>deliverable_cid</code>: "QmDeliverable123" • <code>submission_notes</code>: "Done!" • Contract ID: <code>ct_001</code>, Milestone index: 0
Expected Result	• HTTP 200 with <code>MilestoneResponse</code> • <code>status</code> == "submitted" • <code>deliverable_cid</code> == "QmDeliverable123" • <code>submission_notes</code> == "Done!"
Pass/Fail Criteria	• PASS if milestone transitions <code>pending</code> → <code>submitted</code> with CID recorded • FAIL if status unchanged or CID not stored

3.8 TC ID: TC-M2-08 — Milestone Approval: Client Approves, ETH Released

Field	Value
Req Mapping	FR-MILE-02, FR-CON-02
Priority	High
Technique	Functional — Payment integration
Precondition	Contract <code>ct_001</code> active; milestone <code>ms_000</code> in <code>submitted</code> state; client authenticated
Test Steps	1. Set milestone status to <code>submitted</code> in PostgreSQL 2. Client calls <code>POST /api/contracts/ct_001/milestones/0/approve</code> 3. Backend verifies milestone is <code>submitted</code> 4. Milestone status updated to <code>approved</code>; <code>approved_at</code> set 5. Notification sent to freelancer 6. DB committed and refreshed
Test Data	• Contract ID: <code>ct_001</code>, Milestone index: 0 • Milestone amount: 500.0 ETH equivalent • Platform fee: 2.5% (12.5 ETH)
Expected Result	• HTTP 200 with <code>MilestoneResponse</code> • <code>status == "approved"</code> • <code>approved_at</code> is not <code>None</code> • All milestones approved → contract status becomes <code>completed</code>
Pass/Fail Criteria	• PASS if milestone approved + timestamp recorded + notification sent • FAIL if status not updated or <code>approved_at</code> is <code>None</code>

3.9 TC ID: TC-M2-09 — Milestone Approval: Double Approve Rejected

Field	Value
Req Mapping	NFR-REL-01, FR-SEC-02
Priority	High
Technique	Security — Double-payment prevention
Precondition	Milestone already approved once
Test Steps	1. Set milestone ms_000 to submitted, approve it (status → approved) 2. Attempt second approval via POST /api/contracts/ct_001/milestones/0/approve 3. Assert HTTP 400 4. Assert error code "MILESTONE_NOT_SUBMITTED" 5. Verify no second payment triggered
Test Data	• Milestone pre-set to approved state • Same client token for both calls
Expected Result	• First approve: HTTP 200 • Second approve: HTTP 400 with MILESTONE_NOT_SUBMITTED • Milestone remains in approved state
Pass/Fail Criteria	• PASS if second approval blocked with 400 • FAIL if second approval succeeds (double-spend vulnerability)

3.10 TC ID: TC-M2-10 — Milestone Approval: All Milestones Complete Contract

Field	Value
Req Mapping	FR-MILE-02, FR-DS-02
Priority	High
Technique	Integration — Full lifecycle
Precondition	Contract <code>ct_001</code> active; both milestones <code>ms_000</code> and <code>ms_001</code> in submitted state; client authenticated
Test Steps	1. Client approves <code>ms_000</code> → HTTP 200 2. Client approves <code>ms_001</code> → HTTP 200 3. Query contract <code>ct_001</code> from PostgreSQL 4. Assert contract status is completed 5. Assert both milestones are approved
Test Data	• Two milestones, each 500.0 • Both pre-set to submitted
Expected Result	• Both approvals return HTTP 200 • Contract status transitions active → completed • Contract completion notification sent
Pass/Fail Criteria	• PASS if contract status is completed after all milestones approved • FAIL if contract remains active after final approval

4 Module M3 — Database & State Management

4.1 TC ID: TC-M3-01 — Contract State Transition: Active → Completed

Field	Value
Req Mapping	FR-DS-01, FR-DS-02, NFR-REL-01
Priority	High
Technique	State machine validation
Precondition	Contract <code>ct_001</code> seeded with status <code>active</code> ; two pending milestones
Test Steps	1. Query contract <code>ct_001</code> — assert initial status is <code>active</code> 2. Approve milestone 0 (set to <code>submitted</code> first) 3. Query contract again — assert status is still <code>active</code> (1 of 2 done) 4. Approve milestone 1 5. Query contract — assert status is now <code>completed</code>
Test Data	• Contract: <code>ct_001</code>, milestones: <code>ms_000</code>, <code>ms_001</code> • State transitions: <code>active</code> → <code>completed</code> (on all-approved)
Expected Result	• Contract remains <code>active</code> until all milestones approved • Transitions to <code>completed</code> exactly when last milestone approved • No intermediate invalid states observed
Pass/Fail Criteria	• PASS if state machine follows: <code>active</code> → <code>completed</code> (only on all-approved) • FAIL if premature transition or skipped state

4.2 TC ID: TC-M3-02 — Milestone State Machine: Pending → Submitted → Approved

Field	Value
Req Mapping	FR-DS-01, FR-DS-02
Priority	High
Technique	State machine validation
Precondition	Milestone <code>ms_000</code> seeded in <code>pending</code> state
Test Steps	1. Query <code>ms_000</code> — assert status is <code>pending</code> 2. Freelancer submits → status becomes <code>submitted</code> 3. Query <code>ms_000</code> — assert status is <code>submitted</code> and <code>submitted_at</code> is set 4. Client approves → status becomes <code>approved</code> 5. Query <code>ms_000</code> — assert status is <code>approved</code> and <code>approved_at</code> is set
Test Data	• Valid transitions: <code>pending</code> → <code>submitted</code> → <code>approved</code> • Invalid transitions: <code>approved</code> → <code>submitted</code> (blocked), <code>pending</code> → <code>approved</code> (blocked)
Expected Result	• Only valid state transitions succeed • Invalid transitions return HTTP 400 • Timestamps recorded at each transition
Pass/Fail Criteria	• PASS if all valid transitions succeed and invalid are rejected • FAIL if out-of-order transitions allowed

4.3 TC ID: TC-M3-03 — Cannot Submit Approved Milestone

Field	Value
Req Mapping	FR-DS-02, FR-SEC-02
Priority	High
Technique	Edge case — Invalid state transition
Precondition	Milestone <code>ms_001</code> pre-set to <code>approved</code> state
Test Steps	1. Freelancer calls <code>POST /api/contracts/ct_001/milestones/1/submit</code> 2. Assert HTTP 400 with "MILESTONE_NOT_PENDING" 3. Milestone status remains <code>approved</code>
Test Data	• Milestone pre-set to <code>approved</code> • Deliverable CID: "QmLate"
Expected Result	• HTTP 400 — cannot re-submit an approved milestone • Milestone unchanged in database
Pass/Fail Criteria	• PASS if submission on approved milestone returns 400 • FAIL if status changes to <code>submitted</code>

4.4 TC ID: TC-M3-04 — Cannot Submit Already Submitted Milestone

Field	Value
Req Mapping	FR-DS-02, FR-SEC-02
Priority	High
Technique	Edge case — Duplicate submission prevention
Precondition	Milestone <code>ms_000</code> pre-set to <code>submitted</code> state
Test Steps	1. Freelancer calls <code>POST /api/contracts/ct_001/milestones/0/submit</code> 2. Assert HTTP 400 with "MILESTONE_NOT_PENDING"
Test Data	• Milestone pre-set to <code>submitted</code> • Deliverable CID: "QmDup"
Expected Result	• HTTP 400 — milestone already submitted, pending review • No duplicate submission recorded
Pass/Fail Criteria	• PASS if re-submission returns 400 • FAIL if duplicate submission accepted

4.5 TC ID: TC-M3-05 — Approve Pending Milestone Rejected

Field	Value
Req Mapping	FR-DS-02, FR-SEC-02
Priority	High
Technique	Edge case — Invalid state for approval
Precondition	Milestone <code>ms_000</code> in <code>pending</code> state (not yet submitted)
Test Steps	1. Client calls <code>POST /api/contracts/ct_001/milestones/0/approve</code> 2. Assert HTTP 400 with "MILESTONE_NOT_SUBMITTED" 3. Milestone remains <code>pending</code>
Test Data	• Milestone in default <code>pending</code> state (from seed) • Client auth token
Expected Result	• HTTP 400 — cannot approve a milestone that hasn't been submitted • Status unchanged
Pass/Fail Criteria	• PASS if approval of pending milestone returns 400 • FAIL if status changes to approved

4.6 TC ID: TC-M3-06 — Dispute State: Active → Disputed

Field	Value
Req Mapping	FR-DISP-01, FR-DS-02
Priority	High
Technique	State machine validation
Precondition	Contract <code>ct_001</code> in <code>active</code> state; client authenticated; blockchain mock succeeds
Test Steps	1. Client calls <code>POST /api/contracts/ct_001/disputes</code> with reason 2. Backend calls <code>raise_dispute_on_chain()</code> (mocked) 3. Dispute created in PostgreSQL with status <code>open</code> 4. Contract status updated to <code>disputed</code> 5. Notifications sent to all admins
Test Data	• Reason: "Unacceptable quality" • Contract status: <code>active</code>
Expected Result	• HTTP 201 with <code>DisputeResponse</code> • Dispute <code>status == "open"</code>, <code>raised_by == "usr_client1"</code> • Contract <code>status == "disputed"</code>
Pass/Fail Criteria	• PASS if dispute created + contract locked in <code>disputed</code> state • FAIL if contract remains <code>active</code> after dispute raised

4.7 TC ID: TC-M3-07 — Cannot Dispute Completed Contract

Field	Value
Req Mapping	FR-DS-02, FR-SEC-02
Priority	High
Technique	Edge case — Invalid state for dispute
Precondition	Contract <code>ct_001</code> pre-set to <code>completed</code>
Test Steps	1. Client calls <code>POST /api/contracts/ct_001/disputes</code> with reason 2. Assert HTTP 400 3. Assert error code <code>"CONTRACT_COMPLETED"</code>
Test Data	• Contract status: <code>completed</code> • Reason: "Late"
Expected Result	• HTTP 400 — completed contracts cannot be disputed
Pass/Fail Criteria	• PASS if dispute on completed contract returns 400 • FAIL if dispute created on completed contract

4.8 TC ID: TC-M3-08 — Duplicate Dispute Blocked

Field	Value
Req Mapping	FR-DS-02, FR-SEC-02
Priority	Medium
Technique	Edge case — Duplicate prevention
Precondition	Contract <code>ct_001</code> active; existing open dispute already in database
Test Steps	1. Insert existing dispute with status <code>open</code> for <code>ct_001</code> 2. Client calls <code>POST /api/contracts/ct_001/disputes</code> 3. Assert HTTP 400 with <code>"DISPUTE_EXISTS"</code>
Test Data	• Existing dispute: <code>Dispute(contract_id='ct_001', status=open)</code> • New dispute reason: <code>"Another"</code>
Expected Result	• HTTP 400 — only one active dispute per contract allowed
Pass/Fail Criteria	• PASS if duplicate dispute attempt returns 400 • FAIL if second open dispute created

5 Module M4 — Security & RBAC

5.1 TC ID: TC-M4-01 — Unauthenticated Request Blocked

Field	Value
Req Mapping	FR-SEC-01, FR-SEC-02, NFR-SEC-02
Priority	High
Technique	Security — Authentication enforcement
Precondition	Backend running; no auth token provided
Test Steps	1. Call <code>POST /api/contracts/ct_001/milestones/0/approve</code> without <code>Authorization</code> header 2. Assert HTTP 401 3. Verify no milestone state change in database
Test Data	• No <code>Authorization</code> header • Valid contract/milestone IDs
Expected Result	• HTTP 401 — unauthenticated requests rejected • No state mutation in database
Pass/Fail Criteria	• PASS if unauthenticated request returns 401 • FAIL if endpoint accessible without token

5.2 TC ID: TC-M4-02 — Freelancer Cannot Approve Milestones

Field	Value
Req Mapping	FR-SEC-01, FR-MILE-02
Priority	High
Technique	Security — RBAC restriction
Precondition	Milestone <code>ms_000</code> set to <code>submitted</code> ; freelancer authenticated
Test Steps	1. Freelancer calls <code>POST /api/contracts/ct_001/milestones/0/approve</code> 2. Backend checks <code>current_user.role != client</code> 3. Assert HTTP 403 with <code>"CLIENT_ONLY"</code>
Test Data	• Freelancer auth token (<code>usr_freel1</code>) • Milestone in <code>submitted</code> state
Expected Result	• HTTP 403 — only client can approve milestones • Milestone remains <code>submitted</code>
Pass/Fail Criteria	• PASS if freelancer approval returns 403 • FAIL if freelancer can approve (RBAC bypass)

5.3 TC ID: TC-M4-03 — Client Cannot Submit Milestones

Field	Value
Req Mapping	FR-SEC-01, FR-MILE-01
Priority	High
Technique	Security — RBAC restriction
Precondition	Milestone <code>ms_000</code> in <code>pending</code> state; client authenticated
Test Steps	1. Client calls <code>POST /api/contracts/ct_001/milestones/0/submit</code> 2. Backend checks <code>current_user.role != freelancer</code> 3. Assert HTTP 403 with <code>"FREELANCER_ONLY"</code>
Test Data	• Client auth token (<code>usr_client1</code>) • Deliverable CID: <code>"QmBad"</code>
Expected Result	• HTTP 403 — only freelancer can submit milestones • Milestone remains <code>pending</code>
Pass/Fail Criteria	• PASS if client submission returns 403 • FAIL if client can submit work

5.4 TC ID: TC-M4-04 — Non-Freelancer Cannot Submit Milestones

Field	Value
Req Mapping	FR-SEC-01
Priority	High
Technique	Security — Contract party restriction
Precondition	Milestone <code>ms_000</code> in <code>pending</code> state; client authenticated (wrong party)
Test Steps	1. Client calls <code>POST /api/contracts/ct_001/milestones/0/submit</code> 2. Backend checks <code>role ≠ freelancer</code> 3. Assert HTTP 403 with <code>"FREELANCER_ONLY"</code>
Test Data	• Client auth token (<code>usr_client1</code>) • Contract assigned to <code>usr_freel1</code>
Expected Result	• HTTP 403 — only the assigned freelancer can submit
Pass/Fail Criteria	• PASS if non-freelancer submission returns 403 • FAIL if wrong party can submit

5.5 TC ID: TC-M4-05 — Freelancer Cannot Resolve Disputes

Field	Value
Req Mapping	FR-SEC-01, FR-DISP-02
Priority	High
Technique	Security — Admin-only restriction
Precondition	Open dispute exists for ct_001; freelancer authenticated
Test Steps	1. Insert open dispute for ct_001 2. Freelancer calls POST /api/disputes/{dispute_id}/resolve 3. Assert HTTP 403 with "ADMIN_REQUIRED"
Test Data	• Freelancer auth token • Dispute in open state
Expected Result	• HTTP 403 — only admin can resolve disputes
Pass/Fail Criteria	• PASS if freelancer resolve attempt returns 403 • FAIL if freelancer can resolve disputes

5.6 TC ID: TC-M4-06 — Client Cannot Resolve Disputes

Field	Value
Req Mapping	FR-SEC-01, FR-DISP-02
Priority	High
Technique	Security — Admin-only restriction
Precondition	Open dispute exists for ct_001; client authenticated
Test Steps	1. Insert open dispute for ct_001 2. Client calls POST /api/disputes/{dispute_id}/resolve 3. Assert HTTP 403 with "ADMIN_REQUIRED"
Test Data	• Client auth token (usr_client1) • Dispute in open state
Expected Result	• HTTP 403 — only admin can resolve disputes
Pass/Fail Criteria	• PASS if client resolve attempt returns 403 • FAIL if client can resolve disputes

5.7 TC ID: TC-M4-07 — Freelancer Raises Dispute

Field	Value
Req Mapping	FR-DISP-01, FR-SEC-01
Priority	High
Technique	Functional — Contract party validation
Precondition	Contract <code>ct_001</code> active; freelancer authenticated; blockchain mock succeeds
Test Steps	1. Freelancer calls <code>POST /api/contracts/ct_001/disputes</code> 2. Backend verifies <code>current_user.id == contract.freelancer_id</code> 3. Dispute created; contract status → <code>disputed</code>
Test Data	• Freelancer auth token (<code>usr_freel1</code>) • Reason: <code>"Client ghosting"</code>
Expected Result	• HTTP 201 — freelancer (as contract party) can raise dispute • Dispute <code>status == "open"</code>
Pass/Fail Criteria	• PASS if freelancer dispute creation succeeds (they are a party) • FAIL if freelancer blocked from raising dispute

5.8 TC ID: TC-M4-08 — Password Hashing Verification

Field	Value
Req Mapping	NFR-SEC-01, FR-SEC-02
Priority	High
Technique	White box — Cryptographic verification
Precondition	User seeded with <code>password_hash</code> generated via <code>passlib</code> with <code>bcrypt</code>
Test Steps	1. Verify <code>password_hash</code> field is not plaintext 2. Verify hash starts with <code>\$2b\$</code> (bcrypt prefix) 3. Verify <code>pwd_context.verify("testpassword123", hash)</code> returns <code>True</code> 4. Verify <code>pwd_context.verify("wrongpassword", hash)</code> returns <code>False</code>
Test Data	• Password: <code>"testpassword123"</code> • Hash generated via <code>CryptContext(schemes=["bcrypt"])</code>
Expected Result	• Password stored as bcrypt hash, never plaintext • Correct password verifies; wrong password fails
Pass/Fail Criteria	• PASS if bcrypt prefix present + correct/wrong verification behave as expected • FAIL if plaintext stored or verification logic inverted

6 Module M5 — Workflow API (FastAPI)

6.1 TC ID: TC-M5-01 — Health Check Endpoint

Field	Value
Req Mapping	NFR-AVAIL-02, FR-BE-01
Priority	Medium
Technique	Functional — Basic liveness probe
Precondition	Backend running on port 8000
Test Steps	1. Call <code>GET /api/health</code> 2. Assert HTTP 200 3. Assert response contains <code>status: "ok"</code>
Test Data	None
Expected Result	• HTTP 200 with <code>{"status": "ok"}</code>
Pass/Fail Criteria	• PASS if health returns 200 + status ok • FAIL if endpoint unreachable or non-200

6.2 TC ID: TC-M5-02 — Create Contract API: Full Integration

Field	Value
Req Mapping	FR-BE-01, FR-IPFS-02, FR-CON-01
Priority	High
Technique	Integration — API endpoint
Precondition	Client authenticated; IPFS and blockchain mocked to succeed
Test Steps	1. Client calls <code>POST /api/contracts</code> with complete payload 2. Backend: IPFS upload → CID, blockchain deploy → <code>on_chain_id</code> 3. PostgreSQL: contract + milestones persisted 4. Response: <code>ContractResponse</code> with all fields
Test Data	• Payload: <code>{job_id, freelancer_id, title, total_amount, milestones[]}</code> • Client auth token
Expected Result	• HTTP 201 • Response contains <code>terms_cid, on_chain_id, status, milestones[]</code>
Pass/Fail Criteria	• PASS if contract created with IPFS CID + on-chain ID + milestones • FAIL if HTTP 500 or missing fields

6.3 TC ID: TC-M5-03 — Submit Milestone API: Full Integration

Field	Value
Req Mapping	FR-BE-01, FR-MILE-01
Priority	High
Technique	Integration — API endpoint
Precondition	Contract active; milestone pending; freelancer authenticated; blockchain mocked
Test Steps	1. Freelancer calls <code>POST /api/contracts/{id}/milestones/{index}/submit</code> 2. Backend: verify role + milestone state, call on-chain (mocked), update DB 3. Notification sent to client 4. Response: <code>MilestoneResponse</code> with updated status
Test Data	• <code>deliverable_cid</code>: "QmDeliverable123" • <code>submission_notes</code>: "Done!"
Expected Result	• HTTP 200; status <code>submitted</code>; CID recorded; notification sent
Pass/Fail Criteria	• PASS if submission accepted + DB updated + notification queued • FAIL if HTTP 500 or status unchanged

6.4 TC ID: TC-M5-04 — Approve Milestone API: Full Integration

Field	Value
Req Mapping	FR-BE-01, FR-MILE-02
Priority	High
Technique	Integration — API endpoint
Precondition	Milestone <code>submitted</code> ; client authenticated
Test Steps	1. Client calls <code>POST /api/contracts/{id}/milestones/{index}/approve</code> 2. Backend: verify role + milestone state, update DB, log transition 3. If all milestones approved → contract <code>completed</code> 4. Notification sent to freelancer
Test Data	• Client auth token • Milestone in <code>submitted</code> state
Expected Result	• HTTP 200; status <code>approved</code>; <code>approved_at</code> set
Pass/Fail Criteria	• PASS if approval recorded + notification sent • FAIL if HTTP 500 or status unchanged

6.5 TC ID: TC-M5-05 — Raise Dispute API: Full Integration

Field	Value
Req Mapping	FR-BE-01, FR-DISP-01
Priority	High
Technique	Integration — API endpoint
Precondition	Contract active; client authenticated; blockchain mock succeeds; admin seeded
Test Steps	1. Client calls <code>POST /api/contracts/{id}/disputes</code> 2. Backend: verify party, on-chain dispute (mocked), create DB record 3. Contract status → <code>disputed</code> 4. Notifications sent to all admins
Test Data	• Reason: "Unacceptable quality" • Client auth token
Expected Result	• HTTP 201; dispute <code>status == "open"</code>; contract <code>status == "disputed"</code>
Pass/Fail Criteria	• PASS if dispute created + contract locked + admin notified • FAIL if HTTP 500 or contract not locked

6.6 TC ID: TC-M5-06 — Admin Resolve Dispute: Release to Freelancer

Field	Value
Req Mapping	FR-BE-01, FR-DISP-02
Priority	High
Technique	Integration — Admin workflow
Precondition	Open dispute for <code>ct_001</code> ; admin authenticated; <code>AdminAccount</code> seeded
Test Steps	1. Insert open dispute for <code>ct_001</code> 2. Admin calls <code>POST /api/disputes/{id}/resolve</code> with <code>release_to_freelancer: true</code> 3. Backend: admin check, on-chain resolve (mocked), dispute → resolved 4. Contract → completed; <code>decision = "release"</code>; <code>resolved_by = admin ID</code>
Test Data	• Admin auth token (<code>usr_admin1</code>) • <code>release_to_freelancer: true</code> • <code>resolution_notes: "Client fault"</code>
Expected Result	• HTTP 200; <code>status == "resolved"</code>; <code>decision == "release"</code> • <code>resolved_by == "usr_admin1"</code> • Contract status <code>completed</code>; notifications sent
Pass/Fail Criteria	• PASS if dispute resolved + contract completed + admin recorded • FAIL if resolution fails or contract status unchanged

6.7 TC ID: TC-M5-07 — Admin Resolve Dispute: Refund to Client

Field	Value
Req Mapping	FR-BE-01, FR-DISP-02
Priority	High
Technique	Integration — Admin workflow
Precondition	Open dispute for ct_001; admin authenticated
Test Steps	1. Insert open dispute for ct_001 2. Admin calls POST /api/disputes/{id}/resolve with <code>release_to_freelancer: false</code> 3. Dispute → resolved; decision = "refund" 4. Contract → cancelled; notifications sent
Test Data	• Admin auth token • <code>release_to_freelancer: false</code> • <code>resolution_notes: "Freelancer fault"</code>
Expected Result	• HTTP 200; decision == "refund" • Contract status cancelled
Pass/Fail Criteria	• PASS if refund decision recorded + contract cancelled • FAIL if contract remains disputed or wrong decision

6.8 TC ID: TC-M5-08 — Concurrent Approvals: Race Condition Prevention

Field	Value
Req Mapping	NFR-PERF-02, NFR-REL-01
Priority	High
Technique	Concurrency — Race condition
Precondition	Milestone <code>ms_000</code> set to <code>submitted</code> ; 5 concurrent approval requests prepared
Test Steps	1. Set milestone to <code>submitted</code> 2. Fire 5 concurrent <code>POST /api/contracts/ct_001/milestones/0/approve</code> via <code>asyncio.gather</code> 3. Count HTTP 200 responses 4. Assert exactly 1 succeeds and 4 return 400
Test Data	• 5 concurrent requests, same client token • Milestone amount: 500.0
Expected Result	• Exactly 1 approval succeeds (HTTP 200) • 4 approvals rejected (HTTP 400 with <code>MILESTONE_NOT_SUBMITTED</code>) • No double-spend: only one ETH transfer triggered
Pass/Fail Criteria	• PASS if exactly 1 of 5 concurrent approvals succeeds • FAIL if 2+ succeed (double-spend vulnerability)

6.9 TC ID: TC-M5-09 — Double Approve: Sequential Rejection

Field	Value
Req Mapping	NFR-REL-01, FR-SEC-02
Priority	High
Technique	Security — Sequential double-approval prevention
Precondition	Milestone <code>ms_001</code> set to <code>submitted</code>
Test Steps	1. Client approves <code>ms_001</code> — expect HTTP 200 2. Client approves <code>ms_001</code> again — expect HTTP 400 3. Assert second response has <code>MILESTONE_NOT_SUBMITTED</code>
Test Data	• Sequential (non-concurrent) requests • Same client token
Expected Result	• First: 200; Second: 400 • Milestone remains <code>approved</code> after both calls
Pass/Fail Criteria	• PASS if second approval blocked • FAIL if second approval succeeds

6.10 TC ID: TC-M5-10 — Contract Terms IPFS Upload with Fallback

Field	Value
Req Mapping	FR-IPFS-02, NFR-SCAL-01, NFR-AVAIL-01
Priority	Medium
Technique	Integration — IPFS degraded mode
Precondition	Client authenticated; IPFS returns timeout; blockchain succeeds
Test Steps	1. Client creates contract with valid payload 2. IPFS times out; backend uses fallback <code>terms_cid = f"contract_{id}"</code> 3. Blockchain deploy succeeds 4. Contract saved with fallback CID
Test Data	• IPFS: <code>TimeoutException</code> • Blockchain: mocked success with <code>on_chain_id</code>
Expected Result	• HTTP 201 • <code>terms_cid</code> starts with <code>"contract_"</code> • Contract persisted despite IPFS failure
Pass/Fail Criteria	• PASS if fallback CID used and contract created • FAIL if HTTP 500 or contract not persisted

7 Test Execution Summary

7.1 Test Case Distribution

Module	TC Count	Coverage Area
M1 — Wallet Authentication	4	Nonce, signature, replay, uniqueness
M2 — Smart Contract Engine	10	Create, fund, submit, approve, disputes
M3 — Database & State	8	State machines, transitions, CID linking
M4 — Security & RBAC	8	Auth enforcement, role checks, hashing
M5 — Workflow API	10	Endpoints, integration, concurrency
Total	35	

7.2 Priority Distribution

Priority	Count	Percentage
High	31	88.6%
Medium	4	11.4%

7.3 Technique Distribution

Technique	Count	Percentage
Functional / Integration	14	40.0%
Security / RBAC	12	34.3%
Edge Case	8	22.9%
Concurrency	1	2.9%

7.4 Requirements Coverage

All 29 functional and non-functional requirements from Chapter 3 are covered by at least one test case. The mapping ensures traceability from requirements through test cases to pass/fail criteria.

Requirement Category	Reqs	Covered by TCs
Data Storage (FR-DS)	2	TC-M3-01 to TC-M3-08
IPFS Storage (FR-IPFS)	3	TC-M2-01, TC-M2-02, TC-M2-05, TC-M5-10
Data Sync (FR-SYNC)	2	TC-M3-01, TC-M3-02
Backend Processing (FR-BE)	2	TC-M2-06, TC-M5-01 to TC-M5-10
Security (FR-SEC)	2	TC-M4-01 to TC-M4-08
Wallet Auth (FR-AUTH)	2	TC-M1-01 to TC-M1-04
Smart Contract (FR-CON)	2	TC-M2-01 to TC-M2-06
Milestone (FR-MILE)	2	TC-M2-07 to TC-M2-10
Dispute (FR-DISP)	2	TC-M3-06 to TC-M3-08, TC-M5-05 to TC-M5-07
Performance (NFR-PERF)	2	TC-M5-08, TC-M5-09
Security (NFR-SEC)	2	TC-M4-08, TC-M1-03, TC-M1-04
Scalability (NFR-SCAL)	2	TC-M5-10
Availability (NFR-AVAIL)	2	TC-M2-02, TC-M2-05, TC-M5-10
Reliability (NFR-REL)	2	TC-M2-06, TC-M5-08, TC-M5-09